Systems Analysis of Speculative Decoding on Memory-Constrained Hardware

Praful Prakash Goel

Supervisor: Dr. Ferdous Ahmed Barbhuiya

Department of Computer Science and Engineering
Indian Institute of Information Technology Guwahati

April 2026

Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this report are original and have not been submitted in whole or in part for consideration for any other course, degree, or qualification in this or any other university. This report is my own work and contains nothing that is the outcome of work done in collaboration with others except as specified in the text and Acknowledgments. Parts of the work presented in this report, specifically regarding the performance inversion in KV caching, have been prepared for submission to a peer-reviewed conference or journal as an original research paper titled 'Systems Analysis of Speculative Decoding on Memory-Constrained Hardware'.

Praful Prakash Goel,

Roll: 2301148

Department of Computer Science and Engineering,

Indian Institute of Information Technology Guwahati

Acknowledgement

I would like to extend my heartfelt thanks to my project supervisor, Dr. Ferdous Ahmed Barbhuiya, for their constant support, guidance, and constructive suggestions throughout the development of this project. Their mentorship played an important role in shaping the direction and quality of this work.

I would like to mention that this project was completed independently by me, and it is the result of my own efforts and dedication.

I am also grateful to my institution for providing the necessary facilities and support required to complete this project.

Abstract

This report presents a custom implementation and systems analysis of a speculative decoding inference engine on memory-constrained hardware. While Speculative Decoding aims to accelerate inference by verifying multiple draft tokens in parallel, the overhead of Key-Value (KV) cache management during rejected token rollbacks remains poorly understood on constrained hardware. In this project, a system-level analysis of a custom Speculative Decoding inference engine is conducted, evaluating the throughput trade-offs between compute-bound (cacheless) and memory-bound (cached) configurations using custom 30M and 70M parameter draft models. The empirical benchmarks demonstrate that while a compute-bound configuration utilizing a 70M draft model achieves a peak speedup of 1.95× (52.7 TPS compared to 27.0 TPS baseline), enabling KV Cache introduces severe memory bandwidth bottlenecks during sequence rollbacks, resulting in a performance degradation to 0.86×. Furthermore, cross-model validation using three distinct model families: Pythia, SmolLM, and SmolLM2 confirms these trends, illustrating that the synchronization overhead of KV cache rollbacks scales poorly across varying transformer configuration. These findings indicate that on memory-constrained systems, the overhead of KV cache management during speculative verification outweighs the computational savings, suggesting that cacheless speculative decoding is strictly superior for specific hardware profiles.

Contents

1 Introduction **9**

1.1 Background and Motivation 9

1.2 Project Goals 9

1.3 Interface and Engine Overview 10

1.4 Report Structure 10

2 Background and Related Work **12**

2.1 Introduction to LLM Inference 12

2.2 Theoretical Foundations 12

2.2.1 Compute Bound vs Memory Bound Operations 12

2.2.2 KV Caching Mechanics 13

2.3 Speculative Decoding Mechanics 14

2.4 Survey of Optimization Techniques 15

2.4.1 Recent Advances in Speculative Decoding 15

2.4.2 PagedAttention 15

2.4.3 Limitation of Current Performance Surveys 15

3 System Architecture and Design **17**

3.1 Overview of the Speculative Engine 17

3.2 Component Design 17

3.2.1 The Custom Verification Loop 17

3.2.2 Manual KV Cache Truncation 19

3.3 Data Flow and Execution 19

3.4 Design Decisions 19

4 Implementation Details **20**

4.1 Hardware and Software Environment 20

4.2 Model Selection and Scaling 21

4.2.1 Custom Models 21

4.2.2 Standardized Families 21

4.2.3 Training Methodology and Strategy 22
4.3 Benchmarking Configuration 22
4.4 Inference Parameters and Workload 23
4.5 Implementation Challenges 24
4.5.1 Memory Fragmentation and VRAM Management 24
4.5.2 Convergence-Dependent Token Acceptance 25

5 Evaluation and Results **26**
5.1 Evaluation Metrics 26
5.2 Performance Analysis 27
5.2.1 Comparative Analysis 27
5.2.2 Cross-Architectural Validation (Pythia, SmolLM, SmolLM2) . . . 30
5.3 Performance Insights 31

6 Conclusion and Future Work **33**
6.1 Summary of Contributions 33
6.2 Lessons Learned 33
6.2.1 Hardware-Software Co-design 34
6.2.2 The Impact of Training on Inference 34
6.2.3 Implementation Trade-offs 34
6.3 Future Work 34
6.4 Concluding Remarks 35

List of Figures

2.1 Roofline Model illustrating memory-bound and compute-bound operational regimes. The autoregressive decode phase of LLMs operates in the memory-bound region due to low arithmetic intensity, while the prefill phase is compute-bound. Adapted from Williams et al. [1]. 13

2.2 The Draft-then-Verify pipeline of speculative decoding. In (a), the target model validates all draft tokens in a single parallel pass. In (b), a mismatch at token k forces rejection of all subsequent tokens and triggers a KV cache rollback to maintain consistency. 14

5.1 Baseline inference throughput (measured in TPS) for the isolated target and draft models in standard autoregressive mode. The evaluation compares generation speed with and without KV cache, showing that in standard autoregressive mode, enabling KV cache provides a higher throughput. 27

5.2 Speculative decoding throughput (TPS) as a function of lookahead depth (γ), with (a) and (b) showing performance without and with KV caching, respectively. With KV caching, the 70M draft model achieves peak performance at $\gamma = 5$. Beyond this point, throughput degrades below even the baseline due to verification overhead and cache rollback costs. 28

5.3 Comparative analysis of system speedup S relative to the autoregressive baseline across varying lookahead depths (γ). The results highlight the performance gains achieved by the inference engine using the 70M draft model without KV cache, peaking at a speedup of approximately 1.95× at $\gamma = 7$. The dashed lines indicate configurations with KV caching, which exhibit lower efficiency and faster performance degradation. 29

5.4 Draft model efficiency across varying lookahead depths (γ). (a) illustrates the acceptance rate, while (b) shows the mean number of draft tokens accepted per verification step. The 70M model maintains a stable acceptance rate, driving a higher expected token yield than the 30M model. 29

5.5 Performance evaluation of standardized model families. (a) Comparative analysis of system speedup S across varying lookahead depths (γ) for Pythia, SmolLM and SmolLM2, illustrating the performance inversion when KV caching is enabled. (b) Throughput stress test (Tokens Per Second) across increasing generation lengths to evaluate memory bandwidth constraints. 30

List of Tables

4.1 Model Hyperparameters 21
4.2 Architectural Comparison of Model Families 22
4.3 Multi-phase Training Configuration for the 150M Main Model 22

Chapter 1

Introduction

1.1 Background and Motivation

Large Language Models (LLMs) based on the Transformer architecture [2], exhibit superior performance as compared to earlier architectures such as Long Short-Term Memory [3] and Gated Recurrent Units [4]. Despite their strong modeling capabilities, the inference process of these LLMs is autoregressive in nature in which tokens are generated sequentially, one at a time and is fundamentally limited by memory bandwidth on memory-constrained devices.

One promising approach to accelerate inference in LLMs is Speculative Decoding [5,6], in which two models—a larger target model and a smaller draft model—are used in parallel during token generation. The draft model speculatively generates tokens sequentially, which are then verified by the main target model in a single forward pass. Although this technique performs well on large, server-class GPUs such as NVIDIA A100 or H100 clusters—achieving a $2-3\times$ speedup over the baseline model—using KV cache within speculative decoding on memory-constrained hardware introduces significant overhead due to cache management during rollbacks, an issue that remains underexplored in the literature. This work presents a custom implementation of a speculative decoding inference engine and a system level analysis is performed on the engine under controlled experimental conditions to examine throughput trade-offs between compute-bound (cacheless) and memory-bound (cached) configuration.

1.2 Project Goals

The primary objectives of the project include:

1. Develop and implement a custom speculative decoding inference engine in pure

PyTorch to achieve fine-grained control over the KV cache lifecycle, specifically for profiling token rollbacks.

2. Train and evaluate a suite of custom-architected models, comprising a 150M parameter target model and two draft variants (30M and 70M), to establish a controlled benchmarking baseline [7].
3. Validate the performance of the speculative engine across standardized model families, including Pythia [8], SmolLM [9], and SmolLM2 [10], to ensure architectural generalizability.
4. Quantify and analyze the performance degradation (bottleneck) induced by KV cache management on memory-constrained hardware during the speculative verification phase.

1.3 Interface and Engine Overview

The speculative decoding inference system is organized into four functional layers, spanning from high-level user interaction to the underlying model weights and data structures:

1. **Client Interface Layer:** A Python-based command-line interface that allows for the configuration of inference parameters, such as the lookahead depth (γ) and maximum number of tokens to generate, while displaying real-time metrics like Tokens Per Second (TPS).
2. **Speculative Logic Layer:** The core execution engine implemented in pure PyTorch that manages the draft-then-verify pipeline, coordinating the generation of speculative tokens and their parallel verification by the target model.
3. **KV Cache Management Layer:** A custom-engineered subsystem responsible for the manual "rolling back" of the key-value cache. It ensures the cache is truncated at the exact point of rejection to maintain state consistency.
4. **Model and Data Layer**: This layer encompasses the custom-trained 150M target and 30M/70M draft models trained on the OpenWebText corpus [11]. It also includes the standardized Pythia, SmolLM, and SmolLM2 families used for cross-architectural validation.

1.4 Report Structure

The remainder of this report is organized as follows:

- Chapter 2 discusses the theoretical background of LLM inference, the mechanics of KV caching, and a survey of existing speculative decoding techniques.
- Chapter 3 presents the high-level system architecture and the design decisions behind the custom inference engine.
- Chapter 4 provides detailed implementation information regarding model training, the speculative verification loop, and the hardware environment.
- Chapter 5 evaluates the engine's performance across various model families and analyzes the KV cache bottleneck.
- Chapter 6 concludes the report with a summary of contributions, lessons learned, and potential directions for future work.

Chapter 2

Background and Related Work

2.1 Introduction to LLM Inference

The inference process of LLMs is Autoregressive in nature. Autoregressive decoding is a sequential text generation process where models predict the next token based on all previously generated tokens, ensuring high-quality, coherent output. Its primary limitations include slow inference speed due to token-by-token generation, high computational costs, exposure bias during training, and an inability to revise past tokens.

2.2 Theoretical Foundations

2.2.1 Compute Bound vs Memory Bound Operations

The inference performance of large language models (LLMs) is primarily governed by two hardware constraints: the computational capability of the GPU, typically measured in FLOPS, and the memory bandwidth, which determines how quickly data can be transferred from VRAM to the processing cores. GPU operations can be categorized as either compute-bound or memory-bound.

Compute-bound tasks, such as matrix–matrix multiplications during prompt processing, have high arithmetic intensity and are limited by computational throughput. In contrast, autoregressive generation is largely memory-bound, as each new token requires repeatedly loading model weights from VRAM for relatively simple computations. Consequently, on most hardware, inference speed is constrained more by memory bandwidth than by compute performance.

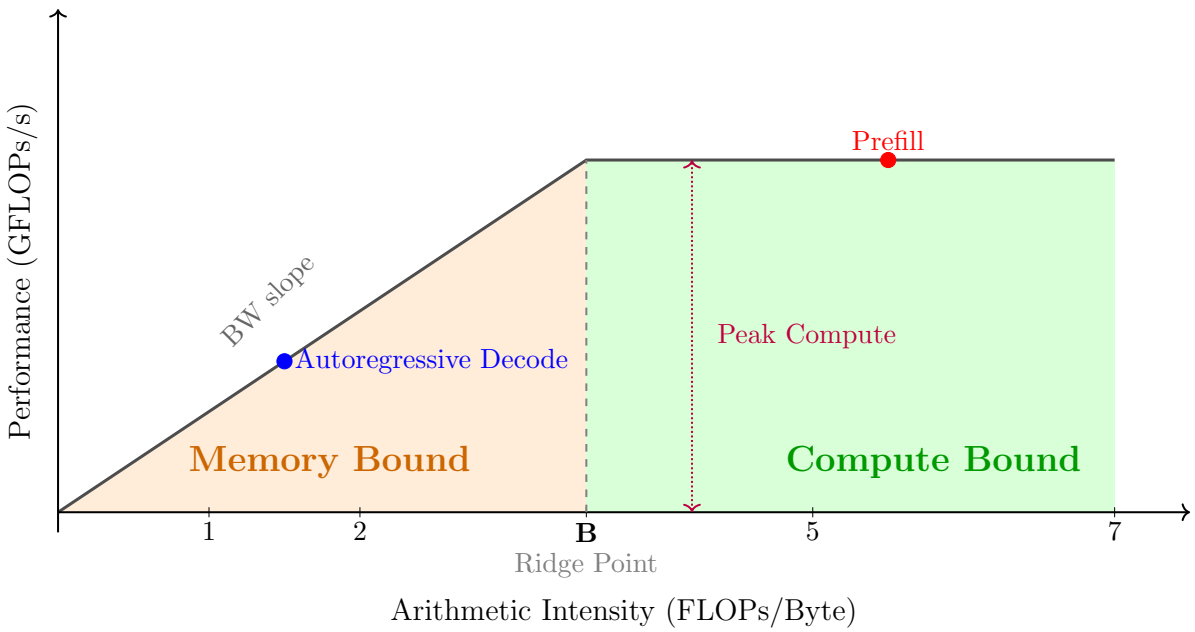


Figure 2.1: Roofline Model illustrating memory-bound and compute-bound operational regimes. The autoregressive decode phase of LLMs operates in the memory-bound region due to low arithmetic intensity, while the prefill phase is compute-bound. Adapted from Williams et al. [1].

2.2.2 KV Caching Mechanics

During autoregressive generation, the model attends to all previously generated tokens to predict the next one, requiring key and value vectors for the entire sequence. Without caching, these are recomputed at every step, resulting in a total cost of $\mathcal{O}(T^2)$ for a sequence of length T, and causing latency to grow with context size.

Key-Value (KV) caching mitigates this by storing previously computed keys and values. After the initial prefill phase, only the new token's vectors are computed while past values are reused, reducing per-token complexity to $\mathcal{O}(T)$ and improving throughput.

This optimization, however, introduces a memory overhead. The KV cache resides in VRAM and scales with sequence length and batch size, requiring $2 \cdot L \cdot H \cdot d$ values per token for a transformer with L layers, H heads, and head dimension d. At long contexts, the cache can become a dominant memory cost. While speculative decoding [5,6] improves efficiency via parallel verification, it requires frequent cache rollbacks when draft tokens are rejected, introducing additional system overhead—especially on memory-constrained hardware.

2.3 Speculative Decoding Mechanics

To address the memory bandwidth limitations of standard autoregressive generation, *speculative decoding* was proposed as a draft-then-verify optimization [5, 6]. Instead of generating one token per forward pass of the large model, the process is split into two stages: drafting and verification.

In the drafting stage, a smaller *draft model* (typically 5–20× smaller) generates γ candidate tokens autoregressively, where γ is the lookahead depth. These tokens are produced efficiently due to the smaller model size. In the verification stage, the larger *target model* evaluates all γ tokens in a single forward pass and determines which to accept. If a mismatch occurs at position k, tokens up to $k-1$ are accepted, the corrected token from the target model is inserted at k, and the remaining speculative tokens are discarded. This mechanism preserves the exact output distribution of the target model while enabling speedup through parallel verification.

From a systems perspective, rejected tokens require rollback of their corresponding KV cache entries, involving memory reallocation and pointer resets. The effectiveness of speculative decoding depends on the expected number of accepted tokens per verification step, given by:

$$E = \sum_{i=1}^{\gamma} P(\text{match}_i) \tag{2.1}$$

where $P(\text{match}_i)$ denotes the probability that the i-th draft token matches the target model. Practical speedup is achieved only when this expected yield offsets both the draft model's compute cost and the memory overhead of cache rollbacks.

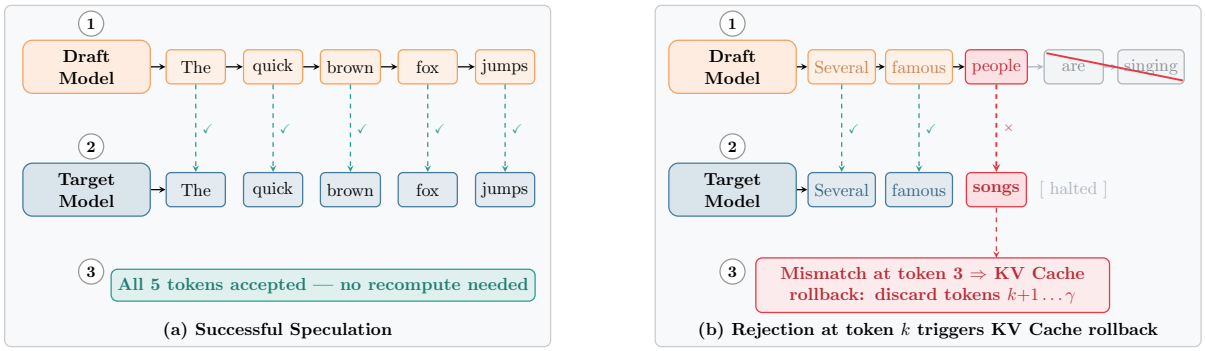


Figure 2.2: The Draft-then-Verify pipeline of speculative decoding. In (a), the target model validates all draft tokens in a single parallel pass. In (b), a mismatch at token k forces rejection of all subsequent tokens and triggers a KV cache rollback to maintain consistency.

2.4 Survey of Optimization Techniques

2.4.1 Recent Advances in Speculative Decoding

Recent approaches such as *Medusa* [12] extend speculative decoding by removing the need for a separate draft model. Instead, Medusa augments the base model with multiple decoding heads that generate several candidate continuations from the final hidden states and verify them in parallel at each step, reducing the number of decoding iterations. Similarly, *staged speculative decoding* [13] introduces a hierarchical pipeline in which a lightweight auxiliary model (e.g., an n-gram model) assists the draft model, further lowering the cost of candidate generation.

While these methods improve speedup and acceptance rates, they are primarily evaluated in high-bandwidth settings where KV cache management overhead is minimal. In particular, existing work often assumes state restoration (e.g., cache truncation and pointer resets) to be negligible. Our analysis challenges this assumption, showing that such overheads become significant on memory-constrained hardware.

2.4.2 PagedAttention

A key limitation of KV caching in existing systems is its large and dynamically resizing memory footprint, often allocated in separate contiguous regions that can lead to fragmentation. *PagedAttention* [14] mitigates this issue by partitioning the KV cache into fixed-size blocks containing keys and values for a set of tokens. These blocks need not be stored contiguously, enabling more flexible allocation and reducing fragmentation.

However, such approaches are primarily designed for high-performance GPUs (e.g., A100/H100) with abundant memory bandwidth. While effective for managing memory layout, they do not account for the I/O overhead introduced by KV cache rollbacks on bandwidth-limited devices. This reveals a key gap: optimizing inference for resource-constrained hardware is fundamentally different from optimizing for server-scale environments.

2.4.3 Limitation of Current Performance Surveys

Prior work on LLM inference efficiency typically treats speculative decoding and KV caching as independent, complementary optimizations. For example, Xia et al. [15] survey speculative decoding methods on hardware such as A100 and RTX 3090, where ample memory allows both a draft model and full KV cache to coexist without contention, implicitly assuming orthogonality. Approaches that do consider their interaction—such as TriForce [16] and MagicDec [17]—leverage KV cache itself as a performance mechanism

by using compressed or sparse representations to reduce memory bandwidth on large-scale, multi-GPU systems.

However, these works do not address scenarios where speculative decoding and standard KV caching must operate under a fixed memory budget. The prevailing assumption is that memory is sufficiently abundant for both techniques to be applied additively. In contrast, our analysis shows that on resource-constrained hardware, the growth of the KV cache reduces the memory available for the draft model, introducing a resource conflict that can degrade performance rather than improve it.

Chapter 3

System Architecture and Design

3.1 Overview of the Speculative Engine

To analyze KV cache-related bottlenecks, a custom speculative decoding framework was built entirely in PyTorch, allowing precise control over cache behavior, especially during rollback operations. Implementing the system in PyTorch ensures complete visibility into the execution process, making it possible to study memory bandwidth effects without interference from optimized backend kernels. This design exposes the internal verification mechanism, enabling accurate tracking of token acceptance rates and the average number of tokens validated per step—metrics that are typically not accessible through standard generation interfaces.

3.2 Component Design

3.2.1 The Custom Verification Loop

The verification loop is the heart of the engine. It follows a "Draft-then-Verify" paradigm where a smaller draft model generates a sequence of K tokens, which are then processed in a single parallel forward pass by the target model. The fundamental operation of the custom speculative decoding engine is outlined in Algorithm 1. In contrast to standard high-level inference frameworks, which abstract away memory management details, our implementation explicitly controls the KV cache state of the draft model. This design enables accurate synchronization during verification: whenever the target model rejects speculative tokens, the system immediately performs a manual truncation of the draft model's cache to maintain consistency.

Algorithm 1 Custom Speculative Decoding with Manual KV Cache Truncation

Input: Main M_p, Draft M_q, Prompt x, Lookahead γ, Max Tokens N
if use_cache **then**
  $M_q.blocks.reset_cache()$
end if
$\hat{x} \leftarrow M_q.generate(x, \gamma)$ {Initial speculation}
while tokens_generated ¡ N **do**
  $L_{in} \leftarrow \text{len}(x)$
  $draft_tokens \leftarrow \hat{x}[L_{in} :]$
  // Parallel Verification Step
  $logits, _ \leftarrow M_p(\hat{x})$
  $verification_logits \leftarrow logits[L_{in} - 1 : L_{in} + \gamma - 1]$
  $main_tokens \leftarrow \text{argmax}(verification_logits)$
  // Verify Drafted Sequence
  $n \leftarrow 0$
  for $i = 0$ **to** $\gamma - 1$ **do**
    if $draft_tokens[i] == main_tokens[i]$ **then**
      $n \leftarrow n + 1$
    else
      break
    end if
  end for
  $correction \leftarrow \text{argmax}(logits[L_{in} + n - 1])$
  $x \leftarrow x \cup draft_tokens[: n] \cup \{correction\}$
  if use_cache **then**
    // The "Memory Wall" Bottleneck
    $valid_len \leftarrow L_{in} + n$
    $M_q.blocks.truncate_cache(valid_len)$
    $draft_in \leftarrow \{correction\}$
  else
    $draft_in \leftarrow x$
  end if
  // Generate Next Speculative Chunk
  $next_chunk \leftarrow M_q.generate(draft_in, \gamma)$
  $\hat{x} \leftarrow x \cup next_chunk[\text{len}(draft_in) :]$
end while
return x

3.2.2 Manual KV Cache Truncation

To maintain the integrity of the autoregressive state, a manual truncation mechanism was developed. Unlike standard inference where the cache grows monotonically, speculative rejection requires the "rolling back" of hidden states. The engine maintains a pointer to the last verified token index; upon rejection, the KV cache tensors are sliced along the sequence dimension to remove invalid speculative states before the next iteration begins.

3.3 Data Flow and Execution

A query follows a cyclic path through the system:

- **Input:** The user provides a prompt and a lookahead depth γ.
- **Drafting:** The draft model populates a speculative buffer.
- **Verification:** The target model validates the buffer in a single batch.
- **Update:** The KV cache is either extended (on success) or truncated (on rejection).
- **Output:** The verified tokens are streamed back to the user interface.

3.4 Design Decisions

The primary design decision was to implement and train custom model architecture and implement the engine in Pure PyTorch rather than using optimized libraries like vLLM or NVIDIA TensorRT-LLM for the following reasons:

1. **Distributional Alignment:** Speculative Decoding efficiency depends heavily on the probability match between Draft and Main models. By training both models on the exact same dataset and tokenizer, I ensured a "Teacher-Student" statistical alignment that is impossible to guarantee with disparate pre-trained models.
2. **White-Box KV Cache Control:** The inference engine requires complex state management (cache rollbacks). A custom implementation provides direct access to the Attention KV Cache tensors, allowing for precise, mathematically correct rejection sampling without fighting high-level library abstractions.
3. **Architectural Flexibility:** Building from scratch allowed for the seamless swapping of disparate model families (e.g., Pythia vs. SmolLM) without being limited by library-specific kernel support.

Chapter 4

Implementation Details

4.1 Hardware and Software Environment

Training was performed on the OpenWebText corpus [11] using NVIDIA A100 Tensor Core GPUs to meet the computational demands of the 150M-parameter target model and the 30M/70M draft models. In contrast, all inference experiments and KV cache profiling were conducted on a local consumer-grade system equipped with an NVIDIA GeForce RTX 3050 Ti GPU with 4GB of VRAM. This separation of training and evaluation environments allows us to study system behavior under realistic deployment constraints rather than idealized server conditions.

In the context of LLM inference, the term memory-constrained does not refer to extremely low-power devices such as microcontrollers, but rather to edge-class accelerators (e.g., mobile GPUs and NPUs) where limited memory bandwidth, rather than compute capacity, becomes the dominant bottleneck. The chosen hardware setup intentionally reflects these constraints, enabling a more faithful examination of performance characteristics encountered in practical edge deployments.

This environment is particularly useful for exposing the overheads associated with KV cache management, especially during rollback operations in speculative decoding. By operating under restricted VRAM and bandwidth conditions, the experiments highlight how memory movement and cache synchronization impact end-to-end latency—effects that are often masked in high-bandwidth server environments.

All experiments were implemented and executed in Python using PyTorch 2.7.1, with CUDA Toolkit 11.8 (cu118) to ensure efficient GPU-accelerated tensor computation. This configuration also allowed fine-grained instrumentation of the inference pipeline, facilitating detailed profiling of both compute and memory behavior.

Table 4.1: Model Hyperparameters

Hyperparameter	Target Model	Draft Model (Small)	Draft Model (Medium)
Parameters	∼150M	∼30M	∼70M
Layers	12	2	6
Attention Heads	12	4	8
Embedding Dim	768	256	512

4.2 Model Selection and Scaling

4.2.1 Custom Models

To evaluate the speculative decoding pipeline, a suite of models was implemented and trained specifically for controlled benchmarking [7]:

- **Target:** 150M parameter Transformer
- **Draft:** 30M and 70M variants

All models were trained on the OpenWebText corpus [11] using the HuggingFace Datasets library [18], ensuring consistency between the draft and target models during verification. Text was tokenized using the GPT-2 tokenizer [19] via the HuggingFace Transformers framework [20]. To maintain compatibility, all models share the same vocabulary size (50,304 tokens), maximum context length (1,024), and architectural features, including Rotary Positional Embeddings (RoPE) and Grouped Query Attention (GQA). A dropout rate of 0.1 was applied uniformly across all configurations. The precise architectural scaling across the target and draft models is detailed in Table 4.1.

The use of two draft models (30M and 70M parameters) enables analysis of the trade-off between model size, token acceptance rate, and overall system performance during speculative decoding.

4.2.2 Standardized Families

In addition to the custom target-draft pairs detailed in Table 4.1, the benchmarking was extended to standardized model families (target-draft pairs):

- Pythia-1B (Target) and Pythia-160M (Draft)
- SmolLM-1.7B (Target) and SmolLM-135M (Draft)
- SmolLM2-1.7B(Target) and SmolLM2-135M (Draft)

These model families were chosen to demonstrate the architecture-agnostic nature of the speculative decoding pipeline and to validate that the observed KV Cache bottleneck

Table 4.2: Architectural Comparison of Model Families

Model Family	Target/Draft	Architecture	Positional Encoding	Attention Type	Training Focus	Release Year
Pythia	1B / 160M	Decoder-only Transformer	Absolute (GPT-style)	Multi-Head Attention	Interpretability, controlled scaling	2023
SmolLM	1.7B / 135M	Efficient Transformer	RoPE	GQA / Optimized Attention	Small-scale efficiency	2024
SmolLM2	1.7B / 135M	Improved Efficient Transformer	RoPE (refined)	GQA (enhanced)	Better data + training efficiency	2025

is a fundamental problem in the transformers architecture rather than family-specific artifacts. The architectural differences between these models is shown in Table 4.2. These differences influence memory access patterns and KV cache behavior, making them relevant for evaluating speculative decoding under bandwidth constraints.

4.2.3 Training Methodology and Strategy

Main Model

The 150M parameter main model was trained using a four-phase curriculum designed to optimize convergence while respecting strict hardware VRAM limits.

Table 4.3: Multi-phase Training Configuration for the 150M Main Model

Phase	Max Iters	Warmup	Batch Size	Accumulation	Learning Rate
1: Warmup	10,000	200	8	32	3×10^{-4}
2: Pre-train	30,000	0	8	32	3×10^{-4}
3: Scaling	50,000	1,000	16	16	6×10^{-5}
4: Convergence	40,000	500	16	16	2×10^{-5}

Draft Models

The draft models were trained in single-phase sessions tailored to their respective parameter counts to ensure they provided sufficiently accurate predictions for the speculative engine.

- **Small Draft Model (30M):** Trained for 40,000 iterations with a batch size of 16 and 1,000 warmup steps.
- **Medium Draft Model (70M):** Trained for 40,000 iterations with 2,000 warmup steps to stabilize the larger gradient updates.

4.3 Benchmarking Configuration

The primary decoding strategy used for benchmarking is deterministic greedy decoding (temperature $T = 0$), wherein the next token is selected as the argmax over the model's output distribution at each step. This choice ensures fully deterministic behavior, enabling direct and fair comparison between baseline autoregressive inference and the speculative decoding pipeline, while preserving strict lossless equivalence in the generated

outputs. The goal of this setup is not to optimize for language modeling quality, perplexity, or generative diversity, but rather to facilitate controlled system-level analysis of performance characteristics—particularly memory bandwidth utilization and KV cache behavior—under constrained hardware conditions.

In addition to greedy decoding, the inference engine supports configurable stochastic decoding strategies, including temperature scaling and top-p (nucleus) sampling, to enable broader experimentation. Under temperature-based sampling, the logits $\mathbf{z}$ are rescaled as $\mathbf{z}' = \mathbf{z}/T$, where $T > 1$ increases randomness and $T < 1$ sharpens the distribution. In nucleus sampling, the next token is selected from the smallest subset of the vocabulary whose cumulative probability exceeds a threshold p. Formally, let V be the vocabulary and $P(x)$ the model's predicted probability distribution. Then the nucleus set V_p is defined as:

$$V_p = \left\{ x \in V \;\middle|\; \sum_{x' \in V_p} P(x') \geq p \right\} \tag{4.1}$$

Sampling is then performed over this truncated distribution, optionally combined with temperature scaling:

$$P_T(x) = \frac{P(x)^{1/T}}{\sum_{x'} P(x')^{1/T}} \tag{4.2}$$

Although these stochastic methods are implemented and exposed as configurable parameters and are useful for general text generation, they introduce variability that complicates reproducibility and performance measurement. Therefore, all reported benchmarks use greedy decoding to ensure consistency, with sampling-based methods included only as optional configurations for extended experimentation.

4.4 Inference Parameters and Workload

All experiments were conducted using a controlled set of inference parameters to ensure reproducibility and fair comparison between baseline and speculative decoding pipelines. For standard generation, the primary configuration includes a maximum generation length of 512 tokens and KV caching enabled by default, with an option to disable caching (–no_cache) for ablation studies. In the speculative decoding pipeline, the lookahead parameter γ is set to 5 unless otherwise specified, representing the number of draft tokens generated per iteration before verification by the target model. This value was chosen as a balance between parallelism and rollback overhead, as larger γ increases potential speedup but also amplifies cache invalidation costs in the event of rejection.

The inference engine supports multiple model configurations, including custom-trained models (150M target with 30M/70M drafts) as well as external model families such as

Pythia, SmolLM, and SmolLM2. This flexibility enables evaluation across varying architectural designs and parameter scales under a unified benchmarking framework. Additional instrumentation is enabled through the –return_stats flag, which records metrics such as token acceptance rate and mean accepted tokens per step, allowing fine-grained analysis of speculative decoding efficiency.

The decoding strategy is primarily greedy decoding ($T = 0$) to ensure deterministic behavior and strict comparability between baseline and speculative runs. While the system also supports stochastic decoding methods such as top-p (nucleus) sampling, these are not used in benchmarking due to the variability they introduce in both output and performance metrics.

For workload generation, a fixed set of representative prompts was used:

- "The capital of France is" (short factual completion)
- "Artificial Intelligence works by" (medium-length explanatory text)
- "The history of the Roman Empire is vast and" (long-form generation)

These prompts were selected to capture varying generation lengths and contextual dependencies, allowing evaluation of system behavior across different inference regimes. Combined with the fixed token budget, this setup ensures consistent stress on both compute and memory subsystems, particularly highlighting the impact of KV cache growth and rollback overhead during speculative decoding.

4.5 Implementation Challenges

The transition from a theoretical speculative decoding framework to a functional engine on consumer-grade hardware presented several significant engineering hurdles. These challenges were compounded by the memory-bound nature of the RTX 3050 Ti.

4.5.1 Memory Fragmentation and VRAM Management

Managing two concurrent KV caches—one for the 150M main model and another for the 30M/70M draft models—within a strict 4GB VRAM budget was the primary obstacle.

- **The Challenge:** Frequent slicing and concatenation of tensors during the speculative "rollback" caused rapid memory fragmentation, leading to RuntimeError: CUDA out of memory even when sufficient total VRAM was technically available.
- **The Solution:** A "Pre-allocation and Masking" strategy was implemented in which instead of dynamically resizing tensors, the maximum context window was pre-allocated at startup. This mirrored the Accumulation Steps logic used in Phase

1 and 2 of the main model training (Section 4.2.3), where high accumulation was necessary to simulate larger batches without overflowing the 4GB buffer.

4.5.2 Convergence-Dependent Token Acceptance

The efficiency of speculative decoding is highly sensitive to the "alignment" between the draft and target models.

- **Training Impact:** Initial implementation attempts using models from Phase 1 of training (Section 4.2.3) resulted in extremely low token acceptance rates ($\alpha < 0.2$), as the models had not yet converged on a consistent probability distribution.
- **The Refinement:** To overcome this, the engine was finalized using weights from Phase 4 (Convergence). This underscored a critical implementation lesson: speculative speedup S is as much a function of the training curriculum (Table 4.3) as it is of the inference algorithm itself.

Chapter 5

Evaluation and Results

5.1 Evaluation Metrics

System-level performance was evaluated using throughput, measured in tokens generated per second (TPS). To quantify the benefit of speculative decoding, the overall speedup S was defined relative to standard autoregressive generation as:

$$S = \frac{\text{TPSspec}}{\text{TPSbase}} \tag{5.1}$$

where TPSspec denotes the throughput achieved by the speculative decoding pipeline, and TPSbase corresponds to the throughput of the baseline target model operating independently.

To further assess the effectiveness of the draft model, two key metrics were considered: the acceptance rate (α) and the average number of tokens accepted per verification step ($\bar{\alpha}$). The acceptance rate is defined as:

$$\alpha = \frac{N_{\text{acc}}}{N_{\text{draft}}} \tag{5.2}$$

where N_{acc} represents the number of draft tokens validated by the target model, and N_{draft} is the total number of tokens proposed by the draft model. Additionally, the mean number of accepted tokens per step is given by:

$$\bar{\alpha} = \frac{N_{\text{acc}}}{N_{\text{steps}}} \tag{5.3}$$

where N_{steps} denotes the total number of verification iterations (i.e., forward passes of the target model). Together, these metrics provide insight into both the efficiency of

speculative generation and the degree of alignment between the draft and target models.

5.2 Performance Analysis

This section analyzes the computational performance of the custom speculative decoding engine. The evaluation focuses on speedup factors and throughput metrics across varying configurations of lookahead depth (γ) and KV cache states.

5.2.1 Comparative Analysis

The initial benchmarks utilized our custom-trained 150M main model paired with 30M and 70M draft variants.

- **Baseline Comparisons:** As shown in Fig. 5.1, the draft models achieve significantly higher throughput than the 150M target model. With KV caching enabled, the 30M and 70M models reach approximately 260 TPS and 84 TPS, respectively, compared to 45 TPS for the target model. This gap, driven by smaller model size and lower memory bandwidth requirements, is essential for speculative decoding to provide net speedup. Additionally, enabling KV cache nearly doubles throughput across all models by avoiding recomputation of key and value tensors for past tokens.

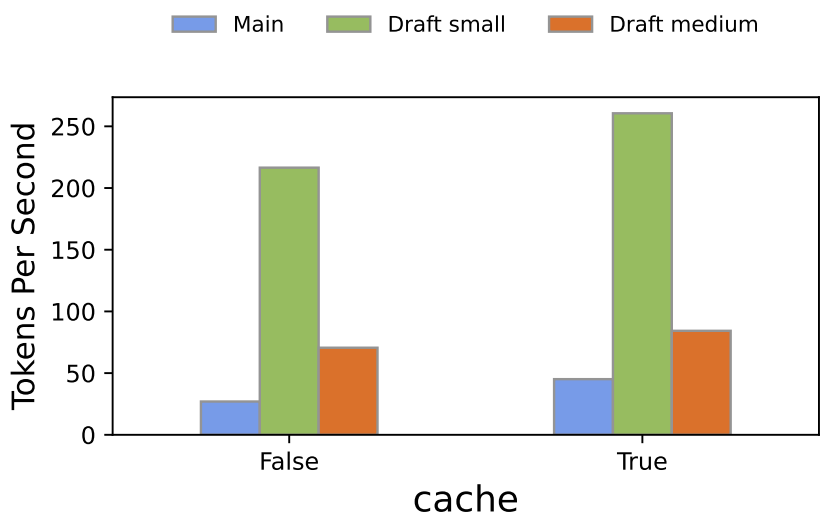


Figure 5.1: Baseline inference throughput (measured in TPS) for the isolated target and draft models in standard autoregressive mode. The evaluation compares generation speed with and without KV cache, showing that in standard autoregressive mode, enabling KV cache provides a higher throughput.

- **Speedup Dynamics:** Fig. 5.2a and 5.2b illustrate the throughput (TPS) of the speculative decoding engine across varying lookahead depths (γ), both without and

with KV cache. In all cases, speculative decoding achieves higher throughput than the corresponding autoregressive baseline. At lower γ, the cache-enabled configuration outperforms the non-cached variant, with the 70M draft model reaching a peak throughput of approximately 58 TPS at $\gamma = 5$. However, as γ increases, throughput declines in both settings, with a significantly sharper drop observed when KV cache is enabled. At higher γ, the cached configuration degrades below even the autoregressive baseline with KV cache, indicating substantial overhead from cache management.

Due to this, the speedup achieved by the engine often drops below their respective baseline models (i.e, S ¡ 1.0) which can be clearly seen in Fig 5.3. The speedup metric is normalized such that $S = 1.0$ represents the autoregressive baseline for each configuration: solid curves correspond to normalization against the non-cached baseline, while dashed curves correspond to the cached baseline. As illustrated in the γ sweep (Fig. 5.3), the non-cached speculative engine achieves a peak speedup of approximately 1.95× for the 70M draft model at $\gamma = 7$. In contrast, enabling the KV cache in same configuration results in a "performance inversion," where the speedup drops to 0.86× relative to the autoregressive baseline, with throughput decreasing from 45.1 TPS to 39.0 TPS.

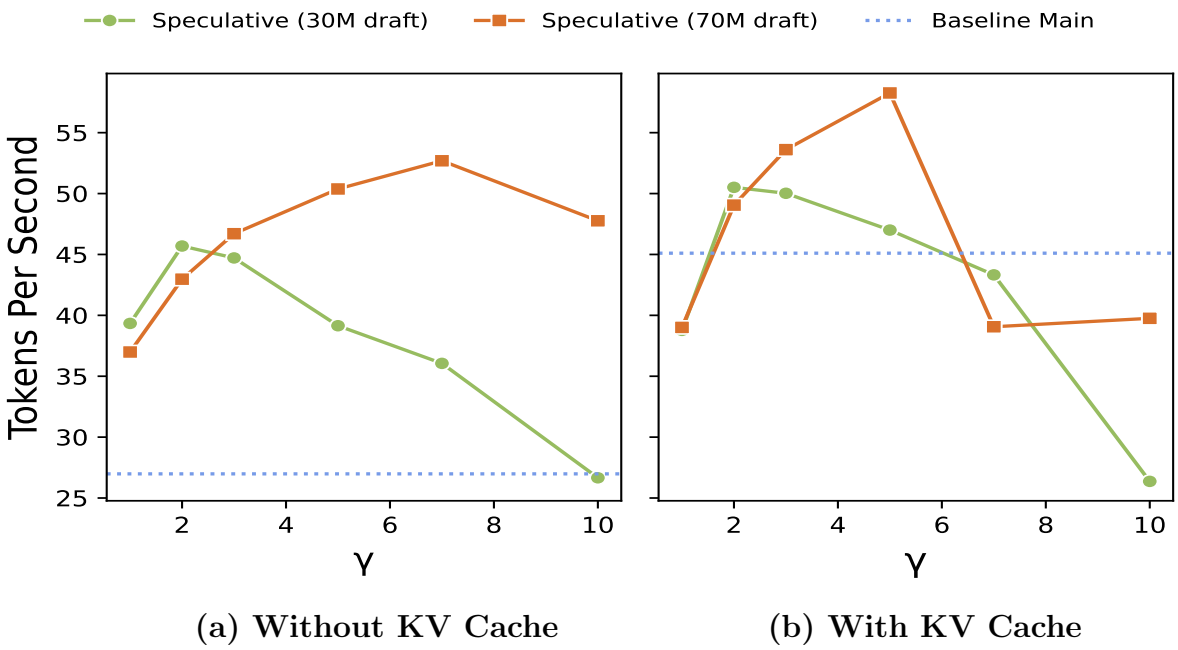


Figure 5.2: Speculative decoding throughput (TPS) as a function of lookahead depth (γ), with (a) and (b) showing performance without and with KV caching, respectively. With KV caching, the 70M draft model achieves peak performance at $\gamma = 5$. Beyond this point, throughput degrades below even the baseline due to verification overhead and cache rollback costs.

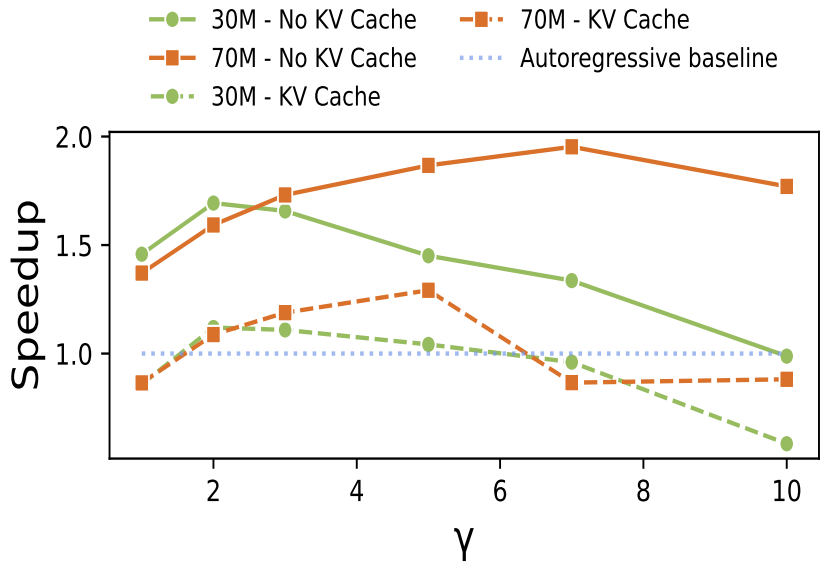
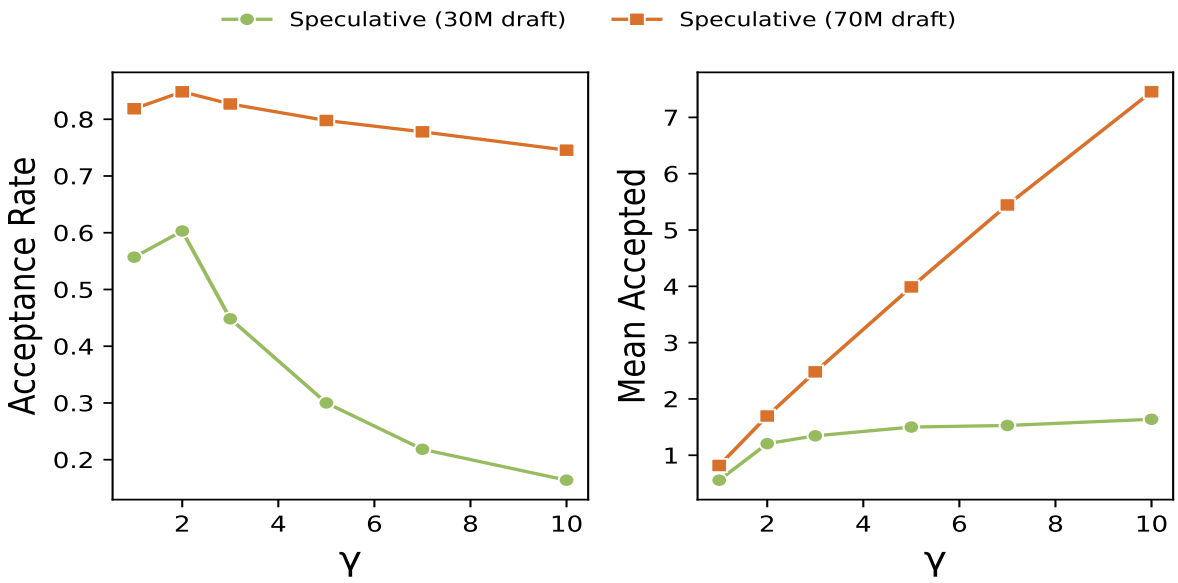


Figure 5.3: Comparative analysis of system speedup S relative to the autoregressive baseline across varying lookahead depths (γ). The results highlight the performance gains achieved by the inference engine using the 70M draft model without KV cache, peaking at a speedup of approximately 1.95× at $\gamma = 7$. The dashed lines indicate configurations with KV caching, which exhibit lower efficiency and faster performance degradation.



(a) Acceptance Rate vs γ **(b) Mean Accepted vs γ**

Figure 5.4: Draft model efficiency across varying lookahead depths (γ). (a) illustrates the acceptance rate, while (b) shows the mean number of draft tokens accepted per verification step. The 70M model maintains a stable acceptance rate, driving a higher expected token yield than the 30M model.

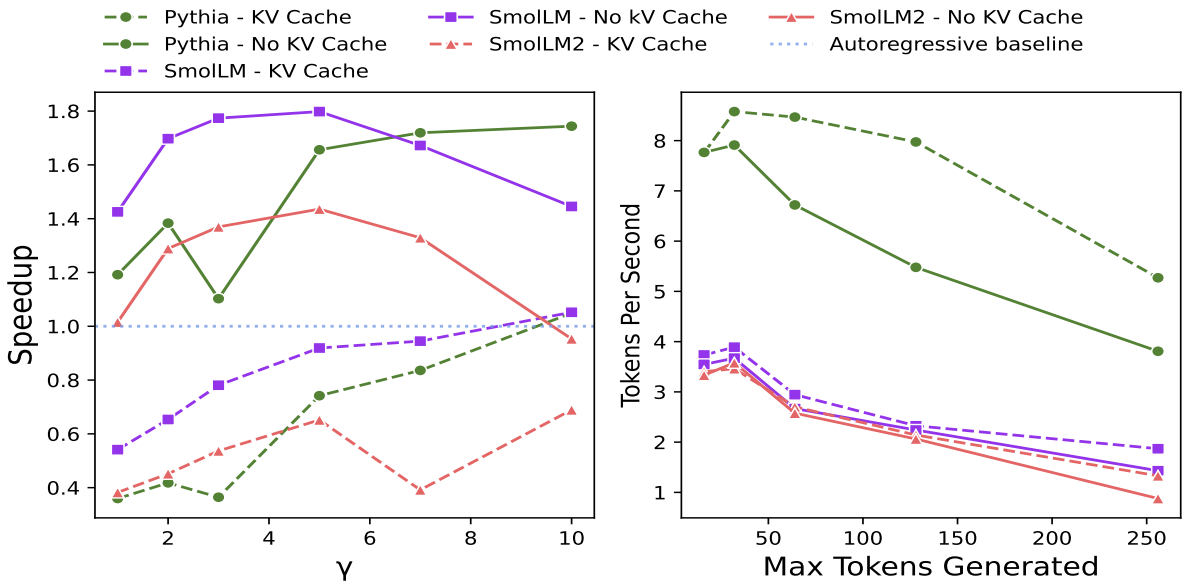
- **Acceptance Rates:** Fig. 5.4 shows the impact of varying the lookahead depth γ

on both the acceptance rate and the average number of draft tokens validated per step. As γ increases, the 70M draft model demonstrates an increase in the average number of accepted tokens, whereas the 30M model shows a decline, indicating its reduced ability to maintain accurate predictions over longer speculative sequences. As a result, the acceptance rate of the 70M model remains relatively stable, while the 30M model experiences a noticeable drop.

From Eq. 2.1, the expected token yield E is determined by the probability of consecutive token agreements, meaning these differences in acceptance behavior directly influence E. This observation aligns with the speedup trends in Fig. 5, where the 70M model sustains higher performance, while the 30M model shows clear degradation at larger γ values.

5.2.2 Cross-Architectural Validation (Pythia, SmolLM, SmolLM2)

To verify the universality of the KV cache bottleneck, the benchmarks were extended to standardized families.



(a) Speedup vs Lookahead depth **(b) Stress Test**

Figure 5.5: Performance evaluation of standardized model families. (a) Comparative analysis of system speedup S across varying lookahead depths (γ) for Pythia, SmolLM and SmolLM2, illustrating the performance inversion when KV caching is enabled. (b) Throughput stress test (Tokens Per Second) across increasing generation lengths to evaluate memory bandwidth constraints.

- **Standardized Speedup:** Fig. 5.5a demonstrates that the performance regression is not an artifact of our custom weights. Across all evaluated models, enabling KV cache within the speculative decoding pipeline results in a marked drop in

performance compared to the non-cached configuration, with the achieved speedup often falling below the corresponding autoregressive baselines. In particular, the SmolLM2 setup reaches a minimum speedup of approximately 0.39× at $\gamma = 7$, while the Pythia configuration drops to about 0.36× at $\gamma = 3$.

- **Throughput Scaling:** Fig. 5.5b depicts how throughput varies with increasing generation length. Although the cached speculative engine shows a slight advantage over its non-cached counterpart at higher token counts, it remains considerably slower than the corresponding autoregressive baselines. Even under its most efficient operating conditions, the speculative pipeline does not reach the baseline throughput ranges of approximately 7=–16 TPS for Pythia and 4—8 TPS for SmolLM and SmolLM2. This indicates that the overhead associated with managing dual KV caches outweighs any benefits, resulting in an overall performance penalty on the evaluated hardware.

5.3 Performance Insights

Key Finding: Contrary to prior work, enabling KV cache within speculative decoding leads to a net performance degradation on memory-constrained hardware, primarily due to the overhead of cache synchronization and rollback operations.

1. **KV Cache Induces Performance Degradation in Speculative Decoding**

   Across all evaluated models, enabling KV cache within the speculative decoding pipeline results in a noticeable decline in performance compared to the non-cached configuration. In several cases, the achieved speedup falls below the corresponding autoregressive baselines. For instance, the SmolLM2 pipeline reaches a minimum speedup of approximately 0.39× at $\gamma = 7$, while the Pythia configuration drops to around 0.36× at $\gamma = 3$. This degradation is primarily attributed to the overhead of KV cache synchronization, particularly during rollback operations where cache tensors must be truncated and memory pointers reset after token rejection.

2. **Rollback Overhead Dominates at Higher Lookahead Depths**

   As the lookahead depth (γ) increases, throughput declines across both cached and non-cached configurations, with a significantly sharper drop observed in the cached setup. Frequent token mismatches at higher γ trigger repeated cache rollbacks, making memory operations a dominant cost. This leads to a counter-intuitive outcome where the cached speculative engine can perform worse than even the baseline autoregressive model.

3. **Non-Cached Speculative Decoding Exhibits Higher Relative Speedup**

An important observation is that the speculative engine without KV cache consistently achieves higher relative speedup compared to its cached counterpart. This behavior can be explained by the *baseline floor effect*: the non-cached autoregressive baseline is slower and more compute-bound, allowing speculative decoding to deliver larger proportional gains. In contrast, when KV cache is enabled, the baseline is already optimized, raising the performance threshold required for speculation to provide additional benefit.

4. **Draft Model Quality Outweighs Raw Throughput**

   Despite having lower standalone throughput, the 70M draft model consistently outperforms the 30M variant within the speculative pipeline. This is due to its higher acceptance rate (α), as it produces predictions that more closely align with the target model. In contrast, the 30M model suffers from more frequent mispredictions, leading to increased rollback and re-verification overhead. This demonstrates that predictive accuracy and model alignment are more critical than raw generation speed when selecting draft models for speculative decoding.

5. **Throughput Scaling Reveals System-Level Bottlenecks**

   As shown in Fig. 5.2, speculative decoding achieves higher throughput than the baseline at lower γ, but performance deteriorates as γ increases. Similarly, Fig. 5.3 highlights that speedup peaks at moderate γ (e.g., $\sim 1.9\times$ for the 70M model) before declining. Additionally, throughput scaling with generation length (Fig. 5.5b) shows that even at optimal conditions, the speculative engine remains below the baseline throughput range (7–16 TPS for Pythia and 4–8 TPS for SmolLM variants), confirming that KV cache management overhead limits performance on memory-constrained hardware.

6. **Acceptance Rate Drives System Efficiency**

   The effectiveness of speculative decoding is closely tied to the acceptance rate and the average number of tokens accepted per step. The 70M model maintains stable acceptance behavior across increasing γ, leading to higher expected token yield and sustained performance. In contrast, the 30M model exhibits declining acceptance rates at larger γ, directly contributing to reduced speedup and increased system overhead.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This project successfully developed and evaluated a custom speculative decoding inference engine tailored for memory-constrained edge hardware. The key contributions of this work include:

- **Modular Inference Engine:** The implementation of a pure PyTorch speculative engine that provides granular control over the KV cache lifecycle and token verification process.
- **Custom Model Curriculum:** The development and training of a 150M parameter main model and 30M/70M draft models using a four-phase convergence strategy to ensure high token acceptance rates.
- **Bottleneck Identification:** A comprehensive profiling of the "Performance Inversion" phenomenon, where state-persistent KV caching leads to a significant speedup deficit—dropping to as low as 0.36×—on consumer-grade GPUs.
- **Cross-Architectural Validation:** Verification of the universal nature of the memory-compute bottleneck across standardized model families, including Pythia, SmolLM, and SmolLM2.

6.2 Lessons Learned

Several critical insights emerged from the design, implementation, and evaluation phases of this project:

6.2.1 Hardware-Software Co-design

The research highlighted that algorithmic speedups in speculative decoding are not guaranteed and are strictly dependent on the underlying hardware's memory bandwidth. On the RTX 3050 Ti, the management of dual KV caches often exceeds the computational savings of parallel verification.

6.2.2 The Impact of Training on Inference

We observed that the token acceptance rate (α) is heavily influenced by the convergence level of the models. Using weights from the final "Convergence" phase of our training curriculum was essential for achieving a speedup $S > 1$ in non-cached configurations.

6.2.3 Implementation Trade-offs

Leveraging pure PyTorch allowed for the necessary transparency to profile low-level cache rollbacks, but it required manual management of memory fragmentation and OOM errors that are typically handled by high-level libraries.

6.3 Future Work

While the current engine exposes critical bottlenecks, several directions for future enhancement remain:

- **Rollback-Aware KV Cache Architectures:** Implementing a system that isolates speculative states from the primary verified cache. By maintaining draft tokens in a separate tentative buffer and only merging them upon successful validation, the engine could bypass the performance-heavy full-cache rollbacks identified in this work. Determining an efficient systems-level implementation for this merging process remains an open challenge.
- **Quantization-Aware Speculation:** Evaluating the engine's performance using 4-bit or 8-bit quantized weights to further alleviate VRAM bandwidth pressure
- **Dynamic Lookahead Depth (γ):** Developing an adaptive algorithm that adjusts the speculation depth in real-time based on the recent token acceptance history.
- **Hardware-Specific Kernel Optimization:** Replacing high-level PyTorch tensor slicing with custom Triton or CUDA kernels to minimize the latency of the KV cache management layer.

6.4 Concluding Remarks

This work conducted a system-level evaluation of a custom speculative decoding inference engine under memory-constrained conditions. The engine was benchmarked across different lookahead depths (γ), draft model sizes, and caching configurations, using a 150M-parameter target model as the baseline. The results show that, in the absence of KV caching, the engine achieves a peak speedup of approximately 1.95×, reaching a throughput of 52.7 TPS.

Contrary to common expectations, enabling KV cache does not consistently improve performance in the speculative setting. Instead, the cache-enabled configuration often underperforms relative to the autoregressive baseline, indicating that the overhead of cache management can outweigh its benefits. As a result, when KV caching is required, standard autoregressive inference may remain the more efficient choice. In contrast, speculative decoding without caching demonstrates stable and meaningful performance gains, highlighting that its effectiveness is maximized when additional memory management overhead is avoided.

Bibliography

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[5] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.

[6] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[7] P. P. Goel, "Speculative decoding models (30m, 70m & 150m)," https://huggingface.co/praful-goel/speculative_decoding_models, 2026, hugging Face Hub.

[8] S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff *et al.*, "Pythia: A suite for analyzing large language models across training and scaling," in *International conference on machine learning*. PMLR, 2023, pp. 2397–2430.

[9] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf, "Smollm - blazingly fast and remarkably powerful," 2024.

[10] L. B. Allal, A. Lozhkov, E. Bakouch, G. M. Blázquez, G. Penedo, L. Tunstall, A. Marafioti, H. Kydlíček, A. P. Lajarín, V. Srivastav, J. Lochner, C. Fahlgren, X.-S. Nguyen, C. Fourrier, B. Burtenshaw, H. Larcher, H. Zhao, C. Zakka, M. Morlon, C. Raffel, L. von Werra, and T. Wolf, "Smollm2: When smol goes big – data-centric training of a small language model," 2025. [Online]. Available: https://arxiv.org/abs/2502.02737

[11] A. Gokaslan and V. Cohen, "Openwebtext corpus," http://Skylion007.github.io/OpenWebTextCorpus, 2019.

[12] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple llm inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.

[13] B. Spector and C. Re, "Accelerating llm inference with staged speculative decoding," *arXiv preprint arXiv:2308.04623*, 2023.

[14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[15] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671, 2024.

[16] H. Sun, Z. Chen, X. Yang, Y. Tian, and B. Chen, "Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding," in *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.

[17] R. Sadhukhan, J. Chen, Z. Chen, V. Tiwari, R. Lai, J. Shi, I. E.-H. Yen, A. May, T. Chen, and B. Chen, "MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[18] Q. Lhoest *et al.*, "Datasets: A community library for natural language processing," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2021, pp. 175–184.

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Technical Report, 2019. [Online]. Available: https://openai.com/research/better-language-models

[20] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2020, pp. 38–45.